Computational Mathematics for Learning and Data Analysys projects, A.Y. 2025/26

Generalities on the projects

Projects are subdivided between “ML” and “no-ML” ones. All projects require the theoretical description, implementation, and testing of one or more algorithms for solving numerical analysis and/or optimization problems. Each project is described with:

- A list (P1), (P2), … of optimization or numerical analysis problems that can be solved using the techniques we have seen in the course. For ML projects, these are the fitting (learning) problems corresponding to one or more Machine Learning models (M1), (M2), … Some details of the problems/models are clearly specified and must be closely adhered to, other details are explicitly left free to choose (possibly with restrictions).

- A list (A1), (A2), … of optimization and/or numerical analysis algorithms suitable for solving the problems (P1), (P2), … (fitting problems corresponding to (M1), (M2), …) that have been seen in the course, or are at least based on the same conceptual tools. Some details of the algorithms are clearly specified and must be closely adhered to, other details are explicitly left free to choose (possibly with restrictions).

All the software developed during the project, as well as the accompanying report, should be strictly of your own making. A few snippets taken from existing software (say, the one provided by the lecturers, or from SourceForge) can be acceptable but they should really be a very minor part. Similarly, copying large parts of existing documents (even with some cosmetic changes aimed at making them superficially different from the original ones) is to be avoided at all costs: direct and explicit citations of existing material (e.g., the statements of the relevant theorems) can be made but the reference to the original source must be clear and explicit. Thus, usage of any existing optimization solvers and/or existing sophisticated linear algebra routines within the project is not allowed unless explicitly permitted by the project statement; in case of doubt contact the lecturers. Please do not make your code/report public during the development and after the conclusion of the exam.

Below, a list of "standard" projects, subdivided among ML and no-ML ones, is provided. These are intended for two-person groups, although one-person groups may be allowed, with reasons, after having gotten explicit permission from the lecturers.

Furthermore, proposals of "wildcard" projects that are different from standard projects are very welcome. Groups willing to perform wildcard projects must send a reasonably detailed proposal (possibly following the scheme used here) to both instructors, who may require any number of changes before approving, and/or not approve at all. In particular, duplicates of standard projects (exact or with minor changes) will *not* be approved.

Since wildcard projects may be more difficult than standard projects (because they include a larger number of problems/models and/or algorithms, or because they consider particularly "nasty" ones), groups proposing them can be formed by more than two students. Of course the number of people in the group must be appropriate to the actual workload of the project, which the lecturers retain the right to ultimately decide upon.

A possible source of wildcard projects that is seen particularly favourably (by one lecturer in particular) is to choose to implement the optimization and/or numerical analysis algorithms within the C++-20 framework SMS++. This typically requires:

- implementing at least one "Block" describing the optimization problem(s), or using/adapting existing ones, and at least one "Solver" implementing the algorithm(s), or adapting/improving existing ones;

- following the tight software development practices envisioned by the project, i.e., a separate git repository, extensive Doxygen-ready documentation of the interface, catering for changes in the model both in the Block and in the Solver, extended, automated and repeatable correctness testing.

Due to the native coarse-grained parallel processing capabilities of SMS++, implementations of parallel algorithms may also be considered.

ML projects

Project 0 - wildcard

(M1), (M2), … are any Machine Learning model for which learning algorithms can be implemented using the techniques we have seen in the course.

(A1), (A2), … are any algorithm (optimization or numerical analysis) suitable for the models (M1), (M2), … that have been seen in the course, or are at least based on the same conceptual tools.

Project 1

(M) is a neural network with topology and activation function of your choice, provided it is differentiable; only smooth regularization (say, L_2) is allowed.

(A1) is a standard momentum descent (heavy ball) approach.

(A2) is an algorithm of the class of Conjugate Gradient methods, cf. also here.

No off-the-shelf solvers allowed.

Project 2

(M) is a neural network with topology and activation function of your choice, provided it is differentiable; only smooth regularization (say, L_2) is allowed.

(A1) is a standard momentum descent (heavy ball) approach.

(A2) is an algorithm of the class of limited-memory quasi-Newton methods, cf. also here and here.

No off-the-shelf solvers allowed.

Project 3

(M1) is a neural network with topology and activation function of your choice (differentiable or not), but mandatory L_1 regularization.

(M2) is a standard L_2 linear regression (least squares).

(A1) is a standard momentum descent (heavy ball) approach applied to (M1).

(A2) is an algorithm of the class of accelerated gradient methods, cf. also here and here, applied to (M1).

(A3) is a basic version of the direct linear least squares solver of your choice (normal equations, QR, or SVD) applied to (M2).

No off-the-shelf solvers allowed.

Project 4

(M1) is a neural network with topology and activation function of your choice (differentiable or not), but mandatory L_1 regularization.

(M2) is a standard L_2 linear regression (least squares).

(A1) is a standard momentum descent (heavy ball) approach applied to (M1).

(A2) is an algorithm of the class of deflected subgradient methods, cf. also here, applied to (M1).

(A3) is a basic version of the direct linear least squares solver of your choice (normal equations, QR, or SVD) applied to (M2).

No off-the-shelf solvers allowed.

Project 5

(M1) is a neural network with topology and activation function of your choice (differentiable or not), but mandatory L_1 regularization.

(A1) is a standard momentum descent (heavy ball) approach.

(A2) is an algorithm of the class of proximal bundle methods.

Use of an off-the-shelf solver for the Master Problem of the bundle method is allowed.

Project 6

(M1) is a neural network with topology of your choice, but mandatory piecewise-linear activation function (of your choice); any regularization is allowed.

(M2) is a standard L_2 linear regression (least squares).

(A1) is a standard momentum descent (heavy ball) approach applied to (M1).

(A2) is an algorithm of the class of deflected subgradient methods, cf. also here, applied to (M1).

(A3) is a basic version of the direct linear least squares solver of your choice (normal equations, QR, or SVD) applied to (M2).

No off-the-shelf solvers allowed.

Project 7

(M1) is a neural network with topology of your choice, but mandatory piecewise-linear activation function (of your choice); any regularization is allowed.

(A1) is a standard momentum descent (heavy ball) approach.

(A2) is an algorithm of the class of level bundle methods.

Use of an off-the-shelf solver for the Master Problem of the bundle method is allowed.

Project 8

(M) is a SVR (support vector regression)-type approach of your choice (in particular, with one or more kernels of your choice).

(A1) is an algorithm of the class of smoothed gradient methods, cf. also here, applied to either the primal or the dual formulation of the SVR.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, save of course for (A2).

Project 9

(M) is a SVR-type approach of your choice (in particular, with one or more kernels of your choice).

(A1) is an algorithm of the class of proximal bundle methods, applied to either the primal or the dual formulation of the SVR.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

Use of an off-the-shelf solver for the Master Problem of the bundle method is allowed.

Project 10

(M) is a SVR-type approach of your choice (in particular, with one or more kernels of your choice).

(A1) is an algorithm of the class of level bundle methods, applied to either the primal or the dual formulation of the SVR.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

Use of an off-the-shelf solver for the Master Problem of the bundle method is allowed.

Project 11

(M) is a SVR-type approach of your choice (in particular, with one or more kernels of your choice).

(A1) is an algorithm of the class of active-set methods, cf. also here, applied to either the primal or the dual formulation of the SVR.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, save of course for (A2).

Project 12

(M) is a SVR-type approach of your choice (in particular, with one or more kernels of your choice).

(A1) is an algorithm of the class of interior-point methods, cf. also here, applied to either the primal or the dual (or both) formulation of the SVR.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, save of course for (A2).

Project 13

(M) is a SVR-type approach of your choice (in particular, with one or more kernels of your choice).

(A1) is an algorithm of the class of gradient projection methods, cf. also here, applied to either the primal or the dual formulation of the SVR.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, save of course for (A2).

Project 14

(M) is a SVR-type approach of your choice (in particular, with one or more kernels of your choice).

(A1) is a Frank-Wolfe type (conditional gradient) algorithm, applied to either the primal or the dual formulation of the SVR.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

Use of an off-the-shelf solver may be allowed for the LP in the Frank-Wolfe approach only, and only after discussing possible ad-hoc approaches and convincingly arguing that they would be too complicated.

Project 15

(M) is a SVR-type approach of your choice (in particular, with one or more kernels of your choice).

(A1) is a dual approach, cf. constrained minimization slides, applied to either the primal or the dual formulation of the SVR, with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved by an algorithm of the class of deflected subgradient methods, cf. also here.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, save of course for (A2).

Project 16

(M) is a SVR-type approach of your choice (in particular, with one or more kernels of your choice).

(A1) is a dual approach, cf. constrained minimization slides, applied to either the primal or the dual formulation of the SVR, with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved by an algorithm of the class of smoothed gradient methods, cf. also here.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, save of course for (A2).

Project 17

(M) is a SVR-type approach of your choice (in particular, with one or more kernels of your choice).

(A1) is a dual approach, cf. constrained minimization slides, applied to either the primal or the dual formulation of the SVR, with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved by an algorithm of the class of proximal bundle methods.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

Use of an off-the-shelf solver for the Master Problem of the bundle method is allowed.

Project 18

(M) is a SVR-type approach of your choice (in particular, with one or more kernels of your choice).

(A1) is a dual approach, cf. constrained minimization slides, applied to either the primal or the dual formulation of the SVR, with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved by an algorithm of the class of level bundle methods.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

Use of an off-the-shelf solver for the Master Problem of the bundle method is allowed.

Project 19

(M) is so-called *extreme learning*, i.e., a neural network with one hidden layer, $y = w\sigma(W_1 x)$, where the weight matrix for the hidden layer W_1 is a fixed random matrix, $\sigma(\cdot)$ is an elementwise activation function of your choice, and the output weight vector w is chosen by solving a L_2 minimization problem $\arg\min_w f(w)$, with $f(w) = \|Xw - y\|_2$.

(A1) is your own implementation of the QR factorization technique, which must obtain linear cost in the largest dimension.

(A2) is *incremental QR*, that is, a strategy in which you update the QR factorization after the addition of a new random feature column x_{n+1} to the feature matrix X. More formally, you are required to write a function that takes as an input the factors of an already-computed factorization $X = QR$ (where Q is stored either directly or via its Householder vectors, at your choice) of $X \in \mathbb{R}^{m \times n}$, and uses them to compute a factorization $\hat{X} = \hat{Q}\hat{R}$ of the matrix $\hat{X} = [X \mid x_{n+1}] \in \mathbb{R}^{m \times (n+1)}$, reusing the work already done. This update should have cost lower than $O(mn^2)$.

No off-the-shelf solvers allowed.

Project 20

(M) is a so-called *extreme learning*, i.e., a neural network with one hidden layer, $y = W_2\sigma(W_1 x)$, where the weight matrix for the hidden layer W_1 is a fixed random matrix, $\sigma(\cdot)$ is an elementwise activation function of your choice, and the output weight matrix W_2 is chosen by solving a linear least-squares problem with L_2 regularization.

(A1) is an algorithm of the class of accelerated gradient methods, cf. also here and here, applied to (M1).

(A2) is a closed-form solution with the normal equations and your own implementation of Cholesky (or LDL) factorization.

No off-the-shelf solvers allowed.

Project 21

(M1) is a so-called *extreme learning*, i.e., a neural network with one hidden layer, $y = W_2\sigma(W_1 x)$, where the weight matrix for the hidden layer W_1 is a fixed random matrix, $\sigma(\cdot)$ is an elementwise activation function of your choice, and the output weight matrix W_2 is chosen by solving a linear least-squares problem, with L_1 regularization.

(M2) as (M1) but with L_2 regularization.

(A1) is an algorithm of the class of deflected subgradient methods, cf. also here.

(A2) is a direct linear least-squares solver of your choice (Cholesky, QR, or SVD), applied to (M2).

No off-the-shelf solvers allowed.

Project 22

(M) is so-called *extreme learning*, i.e., a neural network with one hidden layer, $y = W_2\sigma(W_1 x)$, where the weight matrix for the hidden layer W_1 is a fixed random matrix, $\sigma(\cdot)$ is an elementwise activation function of your choice, and the output weight matrix W_2 is chosen by solving a linear least-squares problem (with L_2 regularization).

(A1) is solution of this linear least-squares problem via thin QR factorization. Your implementation must scale linearly with the largest dimension of the problem.

(A2) is an algorithm of the class of Conjugate Gradient methods.

In addition, for (A2), you are expected to vary the dimension of the hidden layer and study how accuracy and computational time vary based on this choice.

Project 23

(M) is so-called *extreme learning*, i.e., a neural network with one hidden layer, $y = w\sigma(W_1 x)$, where the weight matrix for the hidden layer W_1 is a fixed random matrix, $\sigma(\cdot)$ is an elementwise activation function of your choice, and the output weight vector w is chosen by solving a L_1 minimization problem $\arg\min_w f(w)$, with $f(w) = \|Xw - y\|_1$.

(A1) is *iteratively reweighted least squares*: i.e., an iteration where you solve at each step the L_2 least-squares problem

$$w_{k+1} = \frac{1}{2}\arg\min_w f_k(w), \quad f_k(w) = \|W_k(Xw - y)\|_2^2,$$

where W_k is the diagonal matrix with entries $(W_k)_{ii} = |(Xw_k - y)_i|^{-1/2}$. (You can check that in the limit when $w_k = w_{k+1} = w_*$, f_k and f have the same gradient.) Use a threshold so that the values in W do not get too large if $(Xw_k - y)_i \approx 0$.

(A2) is an algorithm of the class of deflected subgradient methods, cf. also here.

No off-the-shelf solvers allowed.

Project 24

(M) is so-called *extreme learning*, i.e., a neural network with one hidden layer, $y = w\sigma(W_1 x)$, where the weight matrix for the hidden layer W_1 is a fixed random matrix, $\sigma(\cdot)$ is an elementwise activation function of your choice, and the output weight vector w is chosen by solving a L_1 minimization problem $\arg\min_w f(w)$, with $f(w) = \|Xw - y\|_1$.

(A1) is *iteratively reweighted least squares*: i.e., an iteration where you solve at each step the L_2 least-squares problem

$$w_{k+1} = \frac{1}{2}\arg\min_w f_k(w), \quad f_k(w) = \|W_k(Xw - y)\|_2^2,$$

where W_k is the diagonal matrix with entries $(W_k)_{ii} = |(Xw_k - y)_i|^{-1/2}$. (You can check that in the limit when $w_k = w_{k+1} = w_*$, f_k and f have the same gradient.) Use a threshold so that the values in W do not get too large if $(Xw_k - y)_i \approx 0$.

(A2) an algorithm of the class of smoothed gradient methods, cf. also here.

No off-the-shelf solvers allowed.

Project 25

(M) is so-called *extreme learning*, i.e., a neural network with one hidden layer, $y = w\sigma(W_1 x)$, where the weight matrix for the hidden layer W_1 is a fixed random matrix, $\sigma(\cdot)$ is an elementwise activation function of your choice, and the output weight

vector w is chosen by solving a least-squares problem with L_1 regularization $\arg\min_w f(w)$, with $f(w) = \|Xw - y\|_2^2 + \lambda\|w\|_1$.

(A1) is *iteratively reweighted least squares*: i.e., an iteration where you solve at each step the least-squares problem

$$w_{k+1} = \arg\min_w f_k(w), \quad f_k(w) = \frac{1}{2}\|Xw - y\|_2^2 + \lambda\|W_k w\|_2^2$$

where W_k is the diagonal matrix with entries $(W_k)_{ii} = |(w_k)_i|^{-1/2}$. (You can check that in the limit when $w_k = w_{k+1} = w_*$, f_k and f have the same gradient) Use a threshold so that the values in W do not get too large if $(w_k)_i \approx 0$.

(A2) is an algorithm of the class of deflected subgradient methods, cf. also here.

No off-the-shelf solvers allowed.

Project 26

(M) is so-called *extreme learning*, i.e., a neural network with one hidden layer, $y = w\sigma(W_1 x)$, where the weight matrix for the hidden layer W_1 is a fixed random matrix, $\sigma(\cdot)$ is an elementwise activation function of your choice, and the output weight vector w is chosen by solving a least-squares problem with L_1 regularization $\arg\min_w f(w)$, with $f(w) = \|Xw - y\|_2^2 + \lambda\|w\|_1$.

(A1) is *iteratively reweighted least squares*: i.e., an iteration where you solve at each step the least-squares problem

$$w_{k+1} = \arg\min_w f_k(w), \quad f_k(w) = \frac{1}{2}\|Xw - y\|_2^2 + \lambda\|W_k w\|_2^2$$

where W_k is the diagonal matrix with entries $(W_k)_{ii} = |(w_k)_i|^{-1/2}$. (You can check that in the limit when $w_k = w_{k+1} = w_*$, f_k and f have the same gradient) Use a threshold so that the values in W do not get too large if $(w_k)_i \approx 0$.

(A2) an algorithm of the class of proximal bundle methods.

No off-the-shelf solvers allowed.

Project 27

(M) is so-called *extreme learning*, i.e., a neural network with one hidden layer, $y = w\sigma(W_1 x)$, where the weight matrix for the hidden layer W_1 is a fixed random matrix, $\sigma(\cdot)$ is an elementwise activation function of your choice, and the output weight vector w is chosen by solving a least-squares problem with L_1 regularization $\arg\min_w f(w)$, with $f(w) = \|Xw - y\|_2^2 + \lambda\|w\|_1$.

(A1) is *iteratively reweighted least squares*: i.e., an iteration where you solve at each step the least-squares problem

$$w_{k+1} = \arg\min_w f_k(w), \quad f_k(w) = \frac{1}{2}\|Xw - y\|_2^2 + \lambda\|W_k w\|_2^2$$

where W_k is the diagonal matrix with entries $(W_k)_{ii} = |(w_k)_i|^{-1/2}$. (You can check that in the limit when $w_k = w_{k+1} = w_*$, f_k and f have the same gradient) Use a threshold so that the values in W do not get too large if $(w_k)_i \approx 0$.

(A2) an algorithm of the class of level bundle methods.

No off-the-shelf solvers allowed.

Non-ML projects

Project 0 - wildcard

(P1), (P2), … are any optimization or numerical analysis problems for which solution algorithms can be implemented using the techniques we have seen in the course (if more than one problem is selected, they should be related somehow).

(A1), (A2), … are any algorithms (optimization or numerical analysis) suitable for solving the problems (P1), (P2), … that have been seen in the course, or are at least based on the same conceptual tools.

Project 1

(P) is the convex quadratic program on a set K of disjoint simplices

$$\min \left\{ x^T Q x + qx \ : \ \sum_{i \in I^k} x_i = 1 \ \ k \in K \, , \, x \geq 0 \right\}$$

where $x \in \mathbb{R}^n$, the index sets I^k form a partition of the set $\{1, \ldots, n\}$ (i.e., $\cup_{k \in K} I^k = \{1, \ldots, n\}$, and $I^h \cap I^k = \emptyset$ for all h and k), and Q is an $n \times n$ Positive Semidefinite matrix.

(A1) is a solution algorithm based on gradient projection, be it Rosen's or Wolfe's version.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, save of course for (A2).

Project 2

(P) is the convex quadratic program on a set K of disjoint simplices

$$\min \left\{ x^T Q x + qx \ : \ \sum_{i \in I^k} x_i = 1 \ \ k \in K \, , \, x \geq 0 \right\}$$

where $x \in \mathbb{R}^n$, the index sets I^k form a partition of the set $\{1, \ldots, n\}$ (i.e., $\cup_{k \in K} I^k = \{1, \ldots, n\}$, and $I^h \cap I^k = \emptyset$ for all h and k), and Q is an $n \times n$ Positive Semidefinite matrix.

(A1) is a solution algorithm of the class of active-set methods, cf. also here.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solver is allowed, save of course for (A2).

Project 3

(P) is the convex quadratic program on a set K of disjoint simplices

$$\min \left\{ x^T Q x + qx \ : \ \sum_{i \in I^k} x_i = 1 \ \ k \in K \, , \, x \geq 0 \right\}$$

where $x \in \mathbb{R}^n$, the index sets I^k form a partition of the set $\{1, \ldots, n\}$ (i.e., $\cup_{k \in K} I^k = \{1, \ldots, n\}$, and $I^h \cap I^k = \emptyset$ for all h and k), and Q is an $n \times n$ Positive Semidefinite matrix.

(A1) is an algorithm of the class of interior-point methods, cf. also here.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solver is allowed, save of course for (A2).

Project 4

(P) is the convex quadratic program on a set K of disjoint simplices

$$\min \left\{ x^T Q x + qx \ : \ \sum_{i \in I^k} x_i = 1 \ \ k \in K \, , \, x \geq 0 \right\}$$

where $x \in \mathbb{R}^n$, the index sets I^k form a partition of the set $\{1, \ldots, n\}$ (i.e., $\cup_{k \in K} I^k = \{1, \ldots, n\}$, and $I^h \cap I^k = \emptyset$ for all h and k), and Q is an $n \times n$ Positive Semidefinite matrix.

(A1) is an algorithm of the class of conditional gradient (Frank-Wolfe type) methods.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solver is allowed, save of course for (A2): the LP within(A1) must be solved by an ad-hoc approach exploiting the structure of your problem.

Project 5

(P) is the convex quadratic program on a set K of disjoint simplices

$$\min \left\{ x^T Q x + qx \ : \ \sum_{i \in I^k} x_i = 1 \ \ k \in K \, , \, x \geq 0 \right\}$$

where $x \in \mathbb{R}^n$, the index sets I^k form a partition of the set $\{1, \ldots, n\}$ (i.e., $\cup_{k \in K} I^k = \{1, \ldots, n\}$, and $I^h \cap I^k = \emptyset$ for all h and k), and Q is an $n \times n$ Positive Semidefinite matrix.

(A1) is a dual approach, cf. constrained minimization slides, with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved by an algorithm of the class of deflected subgradient methods, cf. also here.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, not even for the solution of the Lagrangian relaxation, save of course for (A2).

Project 6

(P) is the convex quadratic program on a set K of disjoint simplices

$$\min \left\{ x^T Q x + qx \ : \ \sum_{i \in I^k} x_i = 1 \ \ k \in K \, , \, x \geq 0 \right\}$$

where $x \in \mathbb{R}^n$, the index sets I^k form a partition of the set $\{1, \ldots, n\}$ (i.e., $\cup_{k \in K} I^k = \{1, \ldots, n\}$, and $I^h \cap I^k = \emptyset$ for all h and k), and Q is an $n \times n$ Positive Semidefinite matrix.

(A1) is a dual approach, constrained minimization slides, with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved by an algorithm of the class of smoothed gradient methods, cf. also here.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, not even for the solution of the Lagrangian relaxation, save of course for (A2).

Project 7

(P) is the convex quadratic program on a set K of disjoint simplices

$$\min \left\{ x^T Q x + qx \ : \ \sum_{i \in I^k} x_i = 1 \ \ k \in K \, , \, x \geq 0 \right\}$$

where $x \in \mathbb{R}^n$, the index sets I^k form a partition of the set $\{1, \ldots, n\}$ (i.e., $\cup_{k \in K} I^k = \{1, \ldots, n\}$, and $I^h \cap I^k = \emptyset$ for all h and k), and Q is an $n \times n$ Positive Semidefinite matrix.

(A1) is a dual approach, cf. constrained minimization slides, with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved by an algorithm of the class of proximal bundle methods.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

Use of an off-the-shelf solver for the Master Problem of the bundle method is allowed, and of course for (A2), but not for the solution of the Lagrangian relaxation.

Project 8

(P) is the convex quadratic program on a set K of disjoint simplices

$$\min \left\{ x^T Q x + qx \ : \ \sum_{i \in I^k} x_i = 1 \ \ k \in K \, , \, x \geq 0 \right\}$$

where $x \in \mathbb{R}^n$, the index sets I^k form a partition of the set $\{1, \ldots, n\}$ (i.e., $\cup_{k \in K} I^k = \{1, \ldots, n\}$, and $I^h \cap I^k = \emptyset$ for all h and k), and Q is an $n \times n$ Positive Semidefinite matrix.

(A1) is a dual approach, cf. constrained minimization slides, with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved by an algorithm of the class of level bundle methods.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

Use of an off-the-shelf solver for the Master Problem of the bundle method is allowed, and of course for (A2), but not for the solution of the Lagrangian relaxation.

Project 9

(P) is the convex quadratic separable Min-Cost Flow Problem

$$\min \left\{ x^T Q x + qx \,:\, Ex = b\,,\, 0 \le x \le u \right\}$$

where $E \in \mathbb{R}^{n \times m}$ is the node-arc incidence matrix of a directed connected graph with n nodes and m arcs, and $Q \in \mathbb{R}^{m \times m}$ is diagonal and Positive Semidefinite (i.e., $Q = \mathrm{diag}(Q) \ge 0$). You can look, e.g., here for ways to generate meaningful instances of the problem.

(A1) is a solution algorithm of the class of active-set methods, cf. also here.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solver is allowed, save of course for (A2).

Project 10

(P) is the convex quadratic separable Min-Cost Flow Problem

$$\min \left\{ x^T Q x + qx \,:\, Ex = b\,,\, 0 \le x \le u \right\}$$

where $E \in \mathbb{R}^{n \times m}$ is the node-arc incidence matrix of a directed connected graph with n nodes and m arcs, and $Q \in \mathbb{R}^{m \times m}$ is diagonal and Positive Semidefinite (i.e., $Q = \mathrm{diag}(Q) \ge 0$). You can look, e.g., here for ways to generate meaningful instances of the problem.

(A) is an algorithm of the class of interior-point methods, cf. also here.

No off-the-shelf solvers allowed.

Project 11

(P) is the convex quadratic separable Min-Cost Flow Problem

$$\min \left\{ x^T Q x + qx \,:\, Ex = b\,,\, 0 \le x \le u \right\}$$

where $E \in \mathbb{R}^{n \times m}$ is the node-arc incidence matrix of a directed connected graph with n nodes and m arcs, and $Q \in \mathbb{R}^{m \times m}$ is diagonal and Positive Semidefinite (i.e., $Q = \mathrm{diag}(Q) \ge 0$). You can look, e.g., here for ways to generate meaningful instances of the problem.

(A1) must be a solution algorithm of the class of conditional gradient (Frank-Wolfe type) methods.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

Use of an off-the-shelf solver is permitted only for the linear Min-Cost Flow Problem solved at each iteration (and of course for (A2)), but it has to be a specialized solver (not, say, a generic Linear Programming one).

Project 12

(P) is the convex quadratic separable Min-Cost Flow Problem

$$\min \left\{ x^T Q x + qx \,:\, Ex = b\,,\, 0 \leq x \leq u \right\}$$

where $E \in \mathbb{R}^{n \times m}$ is the node-arc incidence matrix of a directed connected graph with n nodes and m arcs, and $Q \in \mathbb{R}^{m \times m}$ is diagonal and Positive Semidefinite (i.e., $Q = \mathrm{diag}(Q) \geq 0$). You can look, e.g., here for ways to generate meaningful instances of the problem.

(A1) must be a dual approach, cf. constrained minimization slides, with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved by an algorithm of the class of deflected subgradient methods, cf. also here.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, save of course for (A2), unless possibly for the solution of the Lagrangian relaxation but only after an analysis that convincingly shows it's a good idea

Project 13

(P) is the convex quadratic separable Min-Cost Flow Problem

$$\min \left\{ x^T Q x + qx \,:\, Ex = b\,,\, 0 \leq x \leq u \right\}$$

where $E \in \mathbb{R}^{n \times m}$ is the node-arc incidence matrix of a directed connected graph with n nodes and m arcs, and $Q \in \mathbb{R}^{m \times m}$ is diagonal and Positive Semidefinite (i.e., $Q = \mathrm{diag}(Q) \geq 0$). You can look, e.g., here for ways to generate meaningful instances of the problem.

(A1) is a dual approach, cf. constrained minimization slides, with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved by an algorithm of the class of smoothed gradient methods, cf. also here.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, save of course for (A2), unless possibly for the solution of the Lagrangian relaxation but only after an analysis that convincingly shows it's a good idea

Project 14

(P) is the convex quadratic separable Min-Cost Flow Problem

$$\min \left\{ x^T Q x + qx \,:\, Ex = b\,,\, 0 \le x \le u \right\}$$

where $E \in \mathbb{R}^{n \times m}$ is the node-arc incidence matrix of a directed connected graph with n nodes and m arcs, and $Q \in \mathbb{R}^{m \times m}$ is diagonal and Positive Semidefinite (i.e., $Q = \mathrm{diag}(Q) \ge 0$). You can look, e.g., here for ways to generate meaningful instances of the problem.

(A1) is a dual approach, constrained minimization slides, with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved by an algorithm of the class of proximal bundle methods.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

Use of an off-the-shelf solver is allowed for the Master Problem of the bundle method, of course for (A2), and possibly also for the solution of the Lagrangian relaxation but only after an analysis that convincingly shows it's a good idea.

Project 15

(P) is the convex quadratic separable Min-Cost Flow Problem

$$\min \left\{ x^T Q x + qx \,:\, Ex = b\,,\, 0 \le x \le u \right\}$$

where $E \in \mathbb{R}^{n \times m}$ is the node-arc incidence matrix of a directed connected graph with n nodes and m arcs, and $Q \in \mathbb{R}^{m \times m}$ is diagonal and Positive Semidefinite (i.e., $Q = \mathrm{diag}(Q) \ge 0$). You can look, e.g., here for ways to generate meaningful instances of the problem.

(A1) is a dual approach, cf. constrained minimization slides, with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved by an algorithm of the class of level bundle methods.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

Use of an off-the-shelf solver is allowed for the Master Problem of the bundle method, of course for (A2), and possibly also for the solution of the Lagrangian relaxation but only after an analysis that convincingly shows it's a good idea.

Project 16

(P) is the problem of estimating the matrix norm $\|A\|_2$ for a (possibly rectangular) matrix $A \in \mathbb{R}^{m \times n}$, using its definition as an (unconstrained) maximum problem.

(A1) is a standard gradient descent (steepest descent) approach.

(A2) is an algorithm of the class of Conjugate Gradient methods, cf. also here.

No off-the-shelf solvers allowed.

Project 17

(P) is the problem of estimating the matrix norm $\|A\|_2$ for a (possibly rectangular) matrix $A \in \mathbb{R}^{m \times n}$, using its definition as an (unconstrained) maximum problem.

(A1) is a standard gradient descent (steepest descent) approach.

(A2) is a quasi-Newton method such as BFGS (one which does not require any approximations of the Hessian of the function).

No off-the-shelf solvers allowed.

Project 18

(P) is the linear least squares problem

$$\min_w \|\hat{X}w - y\|$$

where $\hat{X}$ is the matrix obtained by concatenating the (tall thin) matrix X from the ML-cup dataset by prof. Micheli with a few additional columns containing functions of your choice of the features of the dataset, and y is one of the corresponding output vectors. For instance, if X contains columns $[x1, x2]$, you may add functions such as `log(x1), x1.^2, x1.*x2`, ...

(A1) is an algorithm of the class of Conjugate Gradient methods, cf. also here.

(A2) is thin QR factorization with Householder reflectors (lecture 10 here), in the variant where one does not form the matrix Q, but stores the Householder vectors u_k and uses them to perform (implicitly) products with Q and Q^T.

No off-the-shelf solvers allowed. In particular, you must implement yourself the thin QR factorization, and the computational cost of your implementation should scale linearly with the largest dimension of the matrix X.

Project 19

(P) is the linear least squares problem

$$\min_w \|\hat{X}w - \hat{y}\|$$

where

$$\hat{X} = \begin{bmatrix} X^T \\ \lambda I_m \end{bmatrix}, \quad \hat{y} = \begin{bmatrix} y \\ 0 \end{bmatrix},$$

with $X \in \mathbb{R}^{m \times n}$ the (tall thin) matrix from the ML-cup dataset by prof. Micheli, $\lambda > 0$, and y is a random vector.

(A1) is an algorithm of the class of limited-memory quasi-Newton methods, cf. also here and here.

(A2) is thin QR factorization with Householder reflectors (lecture 10 here) in the variant where one does not form the matrix Q, but stores the Householder vectors u_k and uses them to perform (implicitly) products with Q and Q^T.

No off-the-shelf solvers allowed. In particular you must implement yourself the thin QR factorization, and the computational cost of your implementation should be at most quadratic in m.

Project 20

(P) is the linear least squares problem

$$\min_w \|Xw - y\|$$

where X is the (tall thin) matrix from the ML-cup dataset by prof. Micheli, and y is a random vector.

(A1) is thin QR factorization with Householder reflectors (lecture 10 here)

(A2) is LDL factorization with pivoting applied to the square symmetric linear system

$$\begin{bmatrix} I & X \\ X^T & 0 \end{bmatrix} \begin{bmatrix} r \\ w \end{bmatrix} = \begin{bmatrix} y \\ 0 \end{bmatrix}$$

(*augmented system*).

No off-the-shelf solvers allowed.

Project 21

(P) is a sparse linear system of the form

$$\begin{bmatrix} D & E^T \\ E & 0 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} b \\ c \end{bmatrix}$$

where $D \in \mathbb{R}^{m \times m}$ is a diagonal positive definite matrix and $E \in \mathbb{R}^{n \times m}$ is the node-arc incidence matrix of a given directed graph. These problems arise as the KKT system of the convex quadratic separable Min-Cost Flow Problem, hence you can look, e.g., here for ways to generate meaningful instances of the problem.

(A1) is Conjugate Gradient for linear systems (lecture 38 here), applied to the "reduced" linear system

$$(ED^{-1}E^T)y = ED^{-1}b - c.$$

(Indeed, one can see that eliminating x the two linear systems are equivalent). Your implementation should not require forming the matrix $ED^{-1}E^T$ explicitly, but rely on a callback function which computes the product $ED^{-1}E^Tv$ for a given vector v.

(A2) is GMRES (or MINRES) (lectures 35–36 here), applied to the original system.

No off-the-shelf solvers allowed.

Project 22

(P) is the linear least squares problem

$$\min_w \|\hat{X}w - \hat{y}\|$$

where

$$\hat{X} = \begin{bmatrix} X^T \\ \lambda I \end{bmatrix}, \quad \hat{y} = \begin{bmatrix} y \\ 0 \end{bmatrix},$$

with $X \in \mathbb{R}^{m \times n}$ the (tall thin) matrix from the ML-cup dataset by prof. Micheli, $\lambda > 0$, and y is a random vector.

(A1) is thin QR factorization with Householder reflectors (lecture 10 here).

(A2) is a variant of thin QR (to be determined) in which you make use of the structure of the matrix $\hat{X}$ (and in particular the zeros in the scaled identity block) to reduce the computational cost.

No off-the-shelf solvers allowed. In particular you must implement yourself the thin QR factorization, and the computational cost of your implementation should be at most quadratic in m.

Project 23

(P) is low-rank approximation of a matrix $A \in \mathbb{R}^{m \times n}$, i.e.,

$$\min_{U \in \mathbb{R}^{m \times k}, V \in \mathbb{R}^{n \times k}} \|A - UV^T\|_F$$

(A) is alternating optimization: first take an initial $V = V_0$, and compute $U_1 = \arg\min_U \|A - UV_0^T\|_F$, then use it to compute $V_1 = \arg\min_V \|A - U_1 V^T\|_F$, then $U_2 = \arg\min_U \|A - UV_1^T\|_F$, and so on, until (hopefully) convergence. Use QR factorization to solve these sub-problems.

No off-the-shelf solvers allowed. In particular you must implement yourself QR factorization.

Project 24

(P) is low-rank approximation of a matrix $A \in \mathbb{R}^{m \times n}$, i.e.,

$$\min_{U \in \mathbb{R}^{m \times k}, V \in \mathbb{R}^{n \times k}} \|A - UV^T\|_F$$

(A) is alternating optimization: first take an initial $V = V_0$, and compute $U_1 = \arg\min_U \|A - UV_0^T\|_F$, then use it to compute $V_1 = \arg\min_V \|A - U_1 V^T\|_F$, then $U_2 = \arg\min_U \|A - UV_1^T\|_F$, and so on, until (hopefully) convergence. Use the normal equations with Cholesky factorization to solve these sub-problems.

No off-the-shelf solvers allowed. In particular you must implement yourself Cholesky factorization.

Project 25

(P) is rank-1 approximation of an incomplete matrix: given a subset of its entries $(i_k, j_k, x_k), k = 1, 2, \ldots, p$, find the rank-1 matrix $A = uv^T$ that minimizes

$$\min \sum_{k=1}^{p} (A_{i_k, j_k} - x_k)^2.$$

This method can be used to solve problems of *collaborative filtering*, the most famous of which is the so-called *Netflix prize* (Do not use this dataset though, it would be too large probably.)

(A1) is any optimization method of your choice (on the vectors u, v).

No off-the-shelf solvers allowed.

Project 26

(P) is finding the minimum-L_1 norm solution of an underdetermined linear system, i.e.,

$$\min_{x \text{ s.t. } Ax = y} \|x\|_1,$$

where $A \in \mathbb{R}^{m \times n}$ is short fat, $n > m$.

It is known that when the problem has a sparse solution (i.e., $y = Az$, where z is a vector with $k \ll n$ non-zero elements), in many cases the method recovers the solution $x = z$ (this is known as *compressed sensing*). Experiment with several artificial examples of varying sizes, and see how often this property holds (depending on k).

(A) is any optimization method of your choice.

No off-the-shelf solvers allowed.

Project 27

(P) is a sparse linear system of the form

$$\begin{bmatrix} D & E^T \\ E & 0 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} b \\ c \end{bmatrix}$$

where $D \in \mathbb{R}^{m \times m}$ is a diagonal positive definite matrix (i.e., $D = \text{diag}(D) > 0$) and $E \in \mathbb{R}^{(n-1) \times m}$ is obtained by removing the last row from the node-arc incidence matrix of a given connected directed graph. These problems arise as the KKT system of the convex quadratic separable Min-Cost Flow Problem, hence you can look, e.g., here for ways to generate meaningful instances of the problem.

(A1) is GMRES, and you must solve the internal problems $\min \left\| H_n y - \|b\| e_1 \right\|$ by updating the QR factorization of H_n at each step: given the QR factorization of H_{n-1} computed at the previous step, apply one more orthogonal transformation to compute that of H_n.

(A2) is the same GMRES, but using the so-called *Schur complement preconditioner*

$$P = \begin{bmatrix} D & 0 \\ E & S \end{bmatrix},$$

where S is either $S = ED^{-1}E^T$ or a sparse approximation of it (to obtain it, for instance, replace the smallest off-diagonal entries of S with zeros).

No off-the-shelf solvers allowed.

Project 28

(P) is a sparse linear system of the form

$$\begin{bmatrix} D & E^T \\ E & 0 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} b \\ c \end{bmatrix}$$

where $D \in \mathbb{R}^{m \times m}$ is a diagonal positive definite matrix and $E \in \mathbb{R}^{(n-1) \times m}$ is obtained by removing the last row from the node-arc incidence matrix of a given connected directed graph. These problems arise as the KKT system of the convex quadratic separable Min-Cost Flow Problem, hence you can look, e.g., here for ways to generate meaningful instances of the problem.

(A1) is MINRES, and you must solve the internal problems $\min \left|\left| H_n y - \|b\| e_1 \right|\right|$ by updating the QR factorization of the tridiagonal matrix H_n at each step: given the QR factorization of H_{n-1} computed at the previous step, apply one more orthogonal transformation to compute that of H_n.

(A2) is the same MINRES, but using the so-called *Schur complement preconditioner*

$$P = \begin{bmatrix} D & 0 \\ 0 & S \end{bmatrix},$$

where S is either $S = ED^{-1}E^T$ or a sparse approximation of it (to obtain it, for instance, replace the smallest off-diagonal entries of S with zeros).

No off-the-shelf solvers allowed.

Project 29

(P) is the convex quadratic optimization problem defined on the intersection of a box and an Euclidean ball, i.e.,

$$\min \left\{ x^T Q x + q x \, : \, l \leq x \leq u \, , \, ||x - c|| \leq r \right\}$$

where $x \in \mathbb{R}^n$, q, l, u and c are fixed vectors of $\mathbb{R}^n$, r is a fixed positive real number, and Q is an $n \times n$ Positive Semidefinite matrix.

(A1) is a dual approach, cf. constrained minimization slides, with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved by an algorithm of the class of smoothed gradient methods, cf. also here.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, save of course for (A2), not even for the solution of the Lagrangian relaxation.

Project 30

(P) is the convex quadratic optimization problem defined on the intersection of a box and an Euclidean ball, i.e.,

$$\min \left\{ x^T Q x + q x \, : \, l \leq x \leq u \, , \, ||x - c|| \leq r \right\}$$

where $x \in \mathbb{R}^n$, q, l, u and c are fixed vectors of $\mathbb{R}^n$, r is a fixed positive real number, and Q is an $n \times n$ positive semidefinite matrix.

(A1) is a dual approach, cf. constrained minimization slides, with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved by an algorithm of the class of deflected subgradient methods, cf. also here.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, save of course for (A2), not even for the solution of the Lagrangian relaxation.

Project 31

(P) is the convex quadratic optimization problem defined on the intersection of a box and an Euclidean ball, i.e.,

$$\min \left\{ x^T Q x + q x \, : \, l \leq x \leq u \, , \, ||x - c|| \leq r \right\}$$

where $x \in \mathbb{R}^n$, q, l, u and c are fixed vectors of $\mathbb{R}^n$, r is a fixed positive real number, and Q is an $n \times n$ positive semidefinite matrix.

(A1) is a dual approach [references: constrained minimization slides] with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved by an algorithm of the class of bundle methods.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

Use of an off-the-shelf solver is allowed for the Master Problem of the bundle method and for A2, but not for the solution of the Lagrangian relaxation.

Project 32

(P) is the convex quadratic nonseparable knapsack problem

$$\min \left\{ x^T Q x + q x \, : \, \sum_i a_i x_i \geq b \, , \, l \leq x \leq u \right\}$$

where $x \in \mathbb{R}^n$, q, $0 \leq l < u$ and $a > 0$ are fixed vectors of $\mathbb{R}^n$, b is a fixed positive real number, and Q is an $n \times n$ Positive Semidefinite matrix.

(A1) is a solution algorithm based on gradient projection, be it Rosen's or Wolfe's version.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, save of course for (A2).

Project 33

(P) is the convex quadratic nonseparable knapsack problem

$$\min \left\{ x^T Q x + qx \, : \, \sum_i a_i x_i \geq b \, , \, l \leq x \leq u \right\}$$

where $x \in \mathbb{R}^n$, q, $0 \leq l < u$ and $a > 0$ are fixed vectors of $\mathbb{R}^n$, b is a fixed positive real number, and Q is an $n \times n$ Positive Semidefinite matrix.

(A1) is a solution algorithm of the class of active-set methods, cf. also here.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, save of course for (A2).

Project 34

(P) is the convex quadratic nonseparable knapsack problem

$$\min \left\{ x^T Q x + qx \, : \, \sum_i a_i x_i \geq b \, , \, l \leq x \leq u \right\}$$

where $x \in \mathbb{R}^n$, q, $0 \leq l < u$ and $a > 0$ are fixed vectors of $\mathbb{R}^n$, b is a fixed positive real number, and Q is an $n \times n$ Positive Semidefinite matrix.

(A1) is an algorithm of the class of interior-point methods, cf. also here.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, save of course for (A2).

Project 35

(P) is the convex quadratic nonseparable knapsack problem

$$\min \left\{ x^T Q x + qx \, : \, \sum_i a_i x_i \geq b \, , \, l \leq x \leq u \right\}$$

where $x \in \mathbb{R}^n$, q, $0 \leq l < u$ and $a > 0$ are fixed vectors of $\mathbb{R}^n$, b is a fixed positive real number, and Q is an $n \times n$ Positive Semidefinite matrix.

(A1) is a solution algorithm of the class of conditional gradient (Frank-Wolfe type) methods.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, save of course for (A2): the LP within (A1) must be solved by an ad-hoc approach exploiting the structure of your problem.

Project 36

(P) is the convex quadratic nonseparable knapsack problem

$$\min \left\{ x^T Q x + q x \, : \, \sum_i a_i x_i \geq b \, , \, l \leq x \leq u \right\}$$

where $x \in \mathbb{R}^n$, q, $0 \leq l < u$ and $a > 0$ are fixed vectors of $\mathbb{R}^n$, b is a fixed positive real number, and Q is an $n \times n$ Positive Semidefinite matrix.

(A1) is a dual approach, cf. constrained minimization slides, with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved by an algorithm of the class of deflected subgradient methods, cf. also here.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, save of course for (A2), not even for the solution of the Lagrangian relaxation.

Project 37

(P) is the convex quadratic nonseparable knapsack problem

$$\min \left\{ x^T Q x + q x \, : \, \sum_i a_i x_i \geq b \, , \, l \leq x \leq u \right\}$$

where $x \in \mathbb{R}^n$, q, $0 \leq l < u$ and $a > 0$ are fixed vectors of $\mathbb{R}^n$, b is a fixed positive real number, and Q is an $n \times n$ Positive Semidefinite matrix.

(A1) is a dual approach, cf. constrained minimization slides, with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved by an algorithm of the class of smoothed gradient methods, cf. also here.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, save of course for (A2), not even for the solution of the Lagrangian relaxation.

Project 38

(P) is the convex quadratic nonseparable knapsack problem

$$\min \left\{ x^T Q x + q x \, : \, \sum_i a_i x_i \geq b \, , \, l \leq x \leq u \right\}$$

where $x \in \mathbb{R}^n$, q, $0 \leq l < u$ and $a > 0$ are fixed vectors of $\mathbb{R}^n$, b is a fixed positive real number, and Q is an $n \times n$ Positive Semidefinite matrix.

(A1) is a dual approach, cf. constrained minimization slides, with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved by an algorithm of the class of proximal bundle methods.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

Use of an off-the-shelf solver is allowed for the Master Problem of the bundle method, of course for (A2), but not for the solution of the Lagrangian relaxation.

Project 39

(P) is the convex quadratic nonseparable knapsack problem

$$\min \left\{ x^T Q x + qx \, : \, \sum_i a_i x_i \geq b \, , \, l \leq x \leq u \right\}$$

where $x \in \mathbb{R}^n$, q, $0 \leq l < u$ and $a > 0$ are fixed vectors of $\mathbb{R}^n$, b is a fixed positive real number, and Q is an $n \times n$ Positive Semidefinite matrix.

(A1) is a dual approach, cf. constrained minimization slides, with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved by an algorithm of the class of level bundle methods.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

Use of an off-the-shelf solver is allowed for the Master Problem of the bundle method, of course for (A2), but not for the solution of the Lagrangian relaxation.

Project 40

(P) is the Lasso Least Squares problem

$$\min \left\{ ||Ax - y||_2^2 \, : \, ||x||_1 \leq t \right\}$$

where $x \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$ with $m > n$ and full column rank, $y \in \mathbb{R}^m$, and $t \in \mathbb{R}_{++}$ is a parameter.

(A1) is a solution algorithm based on gradient projection, be it Rosen's or Wolfe's version.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, save of course for (A2).

Project 41

(P) is the Lasso Least Squares problem

$$\min \left\{ ||Ax - y||_2^2 \, : \, ||x||_1 \leq t \right\}$$

where $x \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$ with $m > n$ and full column rank, $y \in \mathbb{R}^m$, and $t \in \mathbb{R}_{++}$ is a parameter.

(A1) is a solution algorithm of the class of active-set methods, cf. also here.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, save of course for (A2).

Project 42

(P) is the Lasso Least Squares problem

$$\min \left\{ ||Ax - y||_2^2 \ : \ ||x||_1 \leq t \right\}$$

where $x \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$ with $m > n$ and full column rank, $y \in \mathbb{R}^m$, and $t \in \mathbb{R}_{++}$ is a parameter.

(A1) is an algorithm of the class of interior-point methods, cf. also here.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, save of course for (A2).

Project 43

(P) is the Lasso Least Squares problem

$$\min \left\{ ||Ax - y||_2^2 \ : \ ||x||_1 \leq t \right\}$$

where $x \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$ with $m > n$ and full column rank, $y \in \mathbb{R}^m$, and $t \in \mathbb{R}_{++}$ is a parameter.

(A1) is an algorithm of the class of conditional gradient (Frank-Wolfe type) methods.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, save of course for (A2): the LP within the algorithm must be solved by an ad-hoc approach exploiting the structure of your problem.

Project 44

(P) is the Lasso Least Squares problem

$$\min \left\{ ||Ax - y||_2^2 \ : \ ||x||_1 \leq t \right\}$$

where $x \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$ with $m > n$ and full column rank, $y \in \mathbb{R}^m$, and $t \in \mathbb{R}_{++}$ is a parameter.

(A1) is a dual approach, cf. constrained minimization slides, with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved either by an algorithm of the class of deflected subgradient methods, cf. also here, or by an ad-hoc method of your choice if reasonable and possible.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, save of course for (A2), not even for the solution of the Lagrangian relaxation.

Project 45

(P) is the Lasso Least Squares problem

$$\min \left\{ ||Ax - y||_2^2 \, : \, ||x||_1 \leq t \right\}$$

where $x \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$ with $m > n$ and full column rank, $y \in \mathbb{R}^m$, and $t \in \mathbb{R}_{++}$ is a parameter.

(A1) is a dual approach, cf. constrained minimization slides, with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved either by an algorithm of the class of smoothed gradient methods, cf. also here, or by an ad-hoc method of your choice if reasonable and possible.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

No off-the-shelf solvers allowed, save of course for (A2), not even for the solution of the Lagrangian relaxation.

Project 46

(P) is the Lasso Least Squares problem

$$\min \left\{ ||Ax - y||_2^2 \, : \, ||x||_1 \leq t \right\}$$

where $x \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$ with $m > n$ and full column rank, $y \in \mathbb{R}^m$, and $t \in \mathbb{R}_{++}$ is a parameter.

(A1) is a dual approach, cf. constrained minimization slides, with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved either by an algorithm of the class of proximal bundle methods, or by an ad-hoc method of your choice if reasonable and possible.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

Use of an off-the-shelf solver is allowed for the Master Problem of the bundle method, and of course for (A2), but not for the solution of the Lagrangian relaxation.

Project 47

(P) is the Lasso Least Squares problem

$$\min \left\{ ||Ax - y||_2^2 \, : \, ||x||_1 \leq t \right\}$$

where $x \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$ with $m > n$ and full column rank, $y \in \mathbb{R}^m$, and $t \in \mathbb{R}_{++}$ is a parameter.

(A1) is a dual approach, cf. constrained minimization slides, with appropriate choices of the constraints to be dualized, where the Lagrangian Dual is solved either by an algorithm of the class of level bundle methods, or by an ad-hoc method of your choice if reasonable and possible.

(A2) is a general-purpose solver applied to an appropriate formulation of the problem.

Use of an off-the-shelf solver is allowed for the Master Problem of the bundle method, and of course for (A2), but not for the solution of the Lagrangian relaxation.

Project 48

(P) is the Nonlinear Convex Continuous Quadratic Minimal Spanning Tree Problem

$$\min \left\{ x^T Q x + qx \ : \ x \in X \right\}$$

where X is the convex hull of the incidence vectors of all possible spanning trees of an undirected connected graph $G = (V, E)$ with n vertices and m edges, and $Q \in \mathbb{R}^{m \times m}$ is Positive Semidefinite (i.e., $Q \succeq 0$).

(A) is a solution algorithm of the class of conditional gradient (Frank-Wolfe type) methods.

No off-the-shelf solver can be used but you are free to use any specialised algorithm for the (linear) Minimal Spanning Tree Problem of your choice.

Optional (suggestion for a three-persons group): also add a solution algorithm solving the problem exactly by an appropriate use of a general-purpose Mixed-Integer Convex Quadratic Program solver.

Project 49

(P) is the Nonlinear Convex Continuous Quadratic Minimal Shortest Path Tree Problem

$$\min \left\{ x^T Q x + qx \ : \ x \in X \right\}$$

where X is the convex hull of the incidence vectors of all possible spanning trees of an directed connected graph $G = (N, A)$ with n vertices and m edges, with prescribed root $r \in N$, and $Q \in \mathbb{R}^{m \times m}$ is Positive Semidefinite (i.e., $Q \succeq 0$).

(A1) is a solution algorithm of the class of conditional gradient (Frank-Wolfe type) methods.

(A2) is the exact approach of applying a general-purpose solver to an appropriate formulation of the problem.

No off-the-shelf solver can be used but you are free to use any specialised algorithm for the (linear) Shortest Path Tree Problem of your choice.

Project 50

(P) is the Constrained Clustering Problem in the L_2 norm: find k centroids for the clusters and assign each of the n inputs to exactly one centroid so as to minimise the sum of the L_2 distances between each point and the assigned centroid, with the constraint that no centroid must be assigned more than a pre-determined number m ($\geq \lceil n/k \rceil$) of points.

(A1) is a heuristic approach obtained by suitably modifying the k-means one.

(A2) is the exact approach of applying a general-purpose solver to an appropriate formulation of the problem.

Off-the-shelf solvers can be used for (A2) (of course) and possibly for the (constrained) inputs assignment step in k-means, but only after appropriate discussion that no better approach is readily available (and even then it should be a specialised solver exploiting the structure of the assignment problem).

Project 51

(P) is (regularized) rank-k matrix completion: given a set of observed entries $\{(A_{ij})\}_{(i,j)\in\Omega}$ of a matrix $A \in \mathbb{R}^{m\times n}$, find factors $U \in \mathbb{R}^{m\times k}$, $V \in \mathbb{R}^{n\times k}$ that minimize

$$\min_{U,V} \sum_{(i,j)\in\Omega} \left(A_{ij} - (UV^T)_{ij}\right)^2 + \lambda(\|U\|_F^2 + \|V\|_F^2).$$

(A1) is Alternating Least Squares (ALS): fix V and compute the optimal U, then fix U and compute the optimal V, iterating until convergence. Solve each least-squares subproblem using your own implementation of thin QR factorization (with Householder reflections).

(A2) is an optimization-based approach of your choice, for example gradient descent.

No off-the-shelf solvers allowed.

Project 52

(P) is a sparse linear system of the form

$$Ax = y$$

where $A \in \mathbb{R}^{n\times n}$ is a sparse symmetric positive definite matrix and $y \in \mathbb{R}^n$ is a given vector.

(A1) is a degree-based reordering routine inspired by the approximate minimum degree (AMD) heuristic. Apply AMD to the sparse system and measure its effect on bandwidth and fill-in.

(A2) is an incomplete Cholesky (IC) preconditioner built with a drop-tolerance, used as a preconditioner for Conjugate Gradient (PCG).

Use of existing software for PCG is allowed but no off-the-shelf solvers for finding IC or AMD are allowed.

Project 53

(P) is a sparse linear system of the form

$$Ax = y$$

where $A \in \mathbb{R}^{n\times n}$ is a sparse symmetric positive definite matrix and $y \in \mathbb{R}^n$ is a given vector.

(A1) is the Reverse Cuthill-McKee (RCM) ordering from the graph adjacency derived from the block matrix. Apply RCM (and compare with the natural ordering) to the sparse system and measure its effect on bandwidth and fill-in.

(A2) is an incomplete Cholesky (IC) preconditioner built with a fixed sparsity pattern, used as a preconditioner for Conjugate Gradient (PCG).

Use of existing software for PCG is allowed but no off-the-shelf solvers for finding IC or RCM are allowed.

Project 54

(P) is the computation of the eigenvalues of a symmetric positive definite matrix $A \in \mathbb{R}^{n \times n}$.

(A1) is the QR algorithm for symmetric matrices, where the QR decomposition is implemented with Householder reflections after a reduction to Hessenberg form, and various shifting strategies are tested (e.g. Wilkinson shift).

(A2) is an existing eigenvalue solver that you can use to check that your implementation is correct.

No off-the-shelf solvers allowed for part (A1).